

MANEJO PROFESIONAL DE DATOS EN SISTEMAS DE MACHINE LEARNING Y REDES NEURONALES

Guía de Referencia Profesional Completa

Basado en el curso de Estrategia del Dato y Machine Learning
Mayo 2026

1. ESTRATEGIA DEL DATO Y FUNDAMENTOS DE IA

La estrategia del dato es el pilar sobre el que se construye cualquier sistema de inteligencia artificial. El principio fundamental es "garbage in, garbage out": la calidad del modelo es directamente proporcional a la calidad de los datos que lo alimentan. Antes de entrenar un solo modelo, una organización debe establecer su arquitectura de datos, sus procesos de gobernanza y sus roles.

1.1 El Patrón Universal de IA

Todo sistema de inteligencia artificial sigue el mismo patrón: Datos → Modelo → Predicción. Este ciclo se repite iterativamente, retroalimentando el sistema con nuevos datos y ajustando el modelo según los resultados obtenidos en producción.

Fase	Descripción	Herramientas clave
Datos	Recolección, limpieza, etiquetado y almacenamiento de datos de calidad	pandas, SQL, Spark, dbt
Modelo	Selección de algoritmo, entrenamiento, validación y ajuste de hiperparámetros	scikit-learn, TensorFlow, PyTorch
Predicción	Despliegue del modelo, inferencia en producción, monitorización y reentrenamiento	Docker, Kubernetes, MLflow, SageMaker

1.2 Tipos de IA y sus aplicaciones

La inteligencia artificial se organiza en varias ramas que evolucionan desde las reglas explícitas hasta el aprendizaje autónomo:

- **Sistemas expertos / IA simbólica:** Reglas explícitas, sin aprendizaje. Adecuados para problemas bien definidos y estables.
- **Machine Learning clásico:** El sistema aprende patrones de datos históricos sin ser programado explícitamente.
- **Deep Learning:** Redes neuronales profundas con múltiples capas de abstracción para datos complejos (imágenes, texto, audio).
- **Modelos fundacionales / LLMs:** Modelos preentrenados en corpus masivos, ajustables a tareas específicas (GPT, BERT, LLaMA).

1.3 Roles en el ecosistema de datos

Rol	Responsabilidad principal	Habilidades técnicas
Chief Data Officer (CDO)	Estrategia global de datos, gobernanza y cumplimiento regulatorio (GDPR)	Liderazgo, visión de negocio, normativas
Data Scientist	Modelado estadístico, experimentación, generación de insights y predicciones	Python, R, estadística, ML, visualización
Data Analyst	Análisis exploratorio, dashboards y reporting para decisiones de negocio	SQL, Power BI, Tableau, estadística descriptiva

Data Engineer	Construcción de pipelines ETL/ELT, infraestructura de datos, data warehouses	SQL, Spark, Airflow, dbt, cloud
ML Engineer	Poner modelos en producción, optimizar rendimiento, gestionar ciclo de vida ML	MLOps, Docker, Kubernetes, APIs, CI/CD
DevOps/SRE	Disponibilidad e infraestructura de los sistemas que soportan los modelos	Cloud, Terraform, Kubernetes, monitorización

1.4 Ética y gobernanza del dato

Un sistema de datos profesional debe cumplir con marcos regulatorios (GDPR en Europa, CCPA en California) y principios éticos de IA responsable. Los datos de personas físicas requieren consentimiento explícito, propósito definido, minimización del dato y derecho al olvido. Los modelos deben ser auditables, justos (sin sesgos discriminatorios) y transparentes en sus decisiones.

- Anonimización y pseudonimización de datos personales.
- Evaluación de sesgos en datos de entrenamiento (representatividad de grupos).
- Explicabilidad de modelos (SHAP, LIME) para decisiones críticas.
- Trazabilidad de datos: linaje completo desde fuente hasta predicción.

2. FUNDAMENTOS DE MACHINE LEARNING

Machine Learning (ML) es la rama de la IA que dota a los sistemas de la capacidad de aprender de los datos sin ser programados explícitamente para cada tarea. Se divide en cuatro paradigmas principales según la naturaleza de los datos y el tipo de señal de aprendizaje disponible.

2.1 Tipos de aprendizaje

Tipo	Datos requeridos	Tarea típica	Algoritmos clave
Supervisado	Datos etiquetados (X, y)	Regresión, Clasificación	Regresión lineal/logística, Random Forest, SVM, Gradient Boosting, Redes neuronales
No supervisado	Datos sin etiquetar (solo X)	Clustering, Reducción de dimensionalidad, Reglas de asociación	K-Means, DBSCAN, PCA, t-SNE, Apriori
Semi-supervisado	Pocos etiquetados + muchos sin etiquetar	Clasificación con datos escasos	Label Propagation, Self-training, Co-training
Por refuerzo	Agente + entorno + recompensa	Control, juegos, robótica, optimización secuencial	Q-Learning, DQN, PPO, Actor-Critic (MDP)

2.2 Aprendizaje supervisado en detalle

Es el paradigma más utilizado en la industria. Requiere un dataset de pares (características X, etiqueta y) y aprende una función f tal que $f(X) \approx y$.

- **Regresión:** Predice una variable continua. Ejemplo: precio de una vivienda, temperatura, ventas futuras. Métricas: MAE, RMSE, R^2 .
- **Clasificación:** Predice una categoría discreta. Ejemplo: spam/no-spam, diagnóstico médico, churn de cliente. Métricas: accuracy, F1, AUC-ROC.

El pipeline estándar de entrenamiento supervisado sigue estos pasos:

- Carga y exploración inicial (EDA): shape, dtypes, describe(), distribuciones, correlaciones.
- Preprocesamiento: limpieza, imputación, codificación, escalado (ver Sección 4).
- División train/val/test: típicamente 70/15/15 o 80/20. Siempre estratificada para clasificación.
- Selección y entrenamiento del modelo: `model.fit(X_train, y_train)`.
- Evaluación y ajuste: métricas sobre val, ajuste de hiperparámetros.
- Evaluación final sobre test (una sola vez). Despliegue.

2.3 Aprendizaje no supervisado: Clustering

El clustering agrupa observaciones similares sin etiquetas previas. Los algoritmos principales son:

- **K-Means:** Requiere especificar k clusters. Minimiza la inercia (suma de distancias al centroide). Sensible a outliers y a la escala → escalar siempre antes. Elbow method para seleccionar k.
- **DBSCAN:** Basado en densidad. No requiere k predefinido. Detecta outliers (puntos de ruido). Parámetros: eps (radio) y min_samples.
- **Clustering jerárquico:** Construye un árbol jerárquico (dendrograma). No requiere k a priori. Computacionalmente costoso para grandes datasets.
- **Evaluación interna:** Evaluación de clustering: Silhouette Score (más alto = mejor, rango -1 a 1), Davies-Bouldin Index (más bajo = mejor).

2.4 Reducción de dimensionalidad

Técnica para reducir el número de variables conservando la mayor información posible. Imprescindible cuando hay muchas columnas correlacionadas (maldición de la dimensionalidad).

- **PCA:** Principal Component Analysis: transformación lineal que maximiza la varianza explicada. Los componentes son ortogonales entre sí. Se usa antes de modelos sensibles a multicolinealidad.
- **t-SNE / UMAP:** t-SNE y UMAP: algoritmos no lineales para visualización en 2D/3D de datos de alta dimensión. No apto para preprocesamiento productivo (no es invertible), pero excelente para exploración.

3. FUNDAMENTOS DE DEEP LEARNING

Deep Learning (DL) es un subconjunto del Machine Learning que utiliza redes neuronales artificiales con múltiples capas (capas ocultas) para aprender representaciones jerárquicas de los datos. Destaca especialmente con datos no estructurados: imágenes, texto, audio y series temporales complejas.

3.1 La neurona artificial (Perceptrón)

La unidad básica de una red neuronal imita la neurona biológica. Recibe múltiples entradas, las pondera con pesos, suma un bias y aplica una función de activación para producir una salida:

$$y = f(\sum(w_i \cdot x_i) + b)$$

Donde:

x_i = entradas
 w_i = pesos (aprendidos durante el entrenamiento)
 b = bias (sesgo, también aprendido)
 f = función de activación no lineal

Una sola neurona es equivalente a una regresión logística. La profundidad (múltiples capas) permite aprender patrones cada vez más abstractos.

3.2 Funciones de activación

La función de activación introduce no linealidad, sin la cual la red sería equivalente a una única transformación lineal independientemente de su profundidad.

Función	Fórmula	Rango	Cuándo usarla	Limitación
ReLU	$\max(0, x)$	$[0, \infty)$	Capas ocultas en general. Estándar en CNN y MLP.	Dying ReLU (neuronas que siempre retornan 0)
Leaky ReLU	$\max(0.01x, x)$	$(-\infty, \infty)$	Alternativa a ReLU para evitar dying ReLU.	Hiperparámetro α adicional
Sigmoid	$1/(1+e^{-x})$	$(0, 1)$	Capa de salida en clasificación binaria.	Vanishing gradient en redes profundas
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	$(-1, 1)$	RNN/LSTM, cuando se necesita salida centrada en 0.	Vanishing gradient, más lenta que ReLU
Softmax	$e^{x_i} / \sum e^{x_j}$	$(0,1)$, suma=1	Capa de salida en clasificación multiclase.	Solo para la capa final
Lineal	$f(x) = x$	$(-\infty, \infty)$	Capa de salida en regresión.	Sin no linealidad (nunca en capas ocultas)

3.3 Backpropagation y Gradient Descent

El entrenamiento de una red neuronal consiste en ajustar los pesos para minimizar una función de pérdida (loss function). El algoritmo de backpropagation calcula el gradiente de la pérdida respecto a cada peso usando la regla de la cadena del cálculo diferencial. Después, gradient descent actualiza los pesos en la dirección contraria al gradiente:

```
# Actualización de pesos (Gradient Descent):  
w = w -  $\eta \cdot \partial L / \partial w$   
  
Donde:  
   $\eta$  (eta) = learning rate (tasa de aprendizaje)  
  L         = función de pérdida (MSE, Cross-Entropy, etc.)  
   $\partial L / \partial w$  = gradiente de la pérdida respecto al peso w
```

Variantes del Gradient Descent:

- **Batch Gradient Descent:** Calcula el gradiente sobre todo el dataset. Preciso pero muy lento para datasets grandes.
- **Stochastic Gradient Descent (SGD):** Actualiza los pesos con cada ejemplo individual. Rápido pero muy ruidoso.
- **Mini-batch Gradient Descent:** Equilibrio entre los dos anteriores. Estándar en producción. Tamaño típico: 32, 64, 128, 256.

Optimizadores avanzados (adaptan el learning rate): Adam (más popular), RMSprop, AdaGrad. Adam combina el momento de SGD con la adaptación del learning rate de RMSprop.

3.4 Arquitecturas de Redes Neuronales Profundas

3.4.1 Redes Neuronales Convolucionales (CNN)

Diseñadas para procesar datos con estructura espacial (imágenes, señales 1D). Introducen la operación de convolución: un kernel (filtro) pequeño se desliza sobre la entrada extrayendo características locales. Las capas se apilan para aprender características de complejidad creciente: bordes → formas → objetos.

- **Capa convolucional:** Operación matemática que aplica un filtro sobre la imagen para detectar patrones locales (bordes, texturas).
- **Capa de Pooling:** Reduce la dimensión espacial conservando las características más relevantes. Tipos: Max Pooling (el valor máximo), Average Pooling (promedio).
- **Flatten:** Transforma el volumen 3D en un vector 1D para pasarlo a capas densas.
- **Capa densa (Dense/FC):** Capa completamente conectada para la clasificación final.

```
# Arquitectura CNN típica en Keras/TensorFlow:  
model = tf.keras.Sequential([  
    Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),  
    MaxPooling2D((2,2)),  
    Conv2D(64, (3,3), activation='relu'),  
    MaxPooling2D((2,2)),  
    Conv2D(128, (3,3), activation='relu'),  
    ...  
])
```

```

    Flatten(),
    Dense(512, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax') # 10 clases
])

```

3.4.2 Redes Neuronales Recurrentes (RNN) y LSTM

Diseñadas para datos secuenciales (series temporales, texto, audio). A diferencia de las redes feedforward, las RNN mantienen un estado oculto (memoria) que se actualiza en cada paso temporal, permitiendo capturar dependencias temporales.

Las RNN simples sufren del problema del vanishing gradient en secuencias largas. Las LSTM (Long Short-Term Memory) lo resuelven con una arquitectura de compuertas:

- **Forget Gate (compuerta de olvido):** Decide qué información del estado anterior se descarta. $\sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
- **Input Gate (compuerta de entrada):** Decide qué nueva información se almacena en la celda.
- **Output Gate (compuerta de salida):** Decide qué parte del estado de la celda se expone como salida.

```

# LSTM en Keras para series temporales:
model = tf.keras.Sequential([
    LSTM(128, return_sequences=True, input_shape=(timesteps, features)),
    LSTM(64),
    Dense(32, activation='relu'),
    Dense(1) # predicción siguiente valor
])

```

3.4.3 Redes Generativas Adversariales (GAN)

Arquitectura de entrenamiento adversarial compuesta por dos redes que compiten entre sí:

- **Generador:** Red que aprende a generar datos sintéticos realistas a partir de ruido aleatorio.
- **Discriminador:** Red que aprende a distinguir datos reales de datos generados. Actúa como juez.

Durante el entrenamiento, el Generador intenta engañar al Discriminador, y este último intenta no ser engañado. El equilibrio de Nash se alcanza cuando el Generador produce datos indistinguibles de los reales. Aplicaciones: generación de imágenes (DALL-E precursor), data augmentation, anonimización de datos, deepfakes (uso ético cuestionable).

4. PREPARACIÓN Y PREPROCESAMIENTO PROFESIONAL DE DATOS

El preprocesamiento es la fase que más tiempo consume en un proyecto de ML (típicamente 60-80% del esfuerzo total) y la que mayor impacto tiene en la calidad del modelo. Un modelo perfecto con datos sucios dará peores resultados que un modelo simple con datos de calidad.

4.1 Exploración y Auditoría inicial del dataset (EDA)

Antes de cualquier transformación, es imprescindible realizar una auditoría completa del dataset para entender su estructura, calidad y distribución:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('datos.csv')

# Dimensiones y tipos
print(df.shape)          # (filas, columnas)
print(df.dtypes)          # tipo de cada columna
print(df.info())          # resumen completo con nulls

# Estadísticas descriptivas
print(df.describe())      # media, std, percentiles de numéricas
print(df.describe(include='object')) # frecuencias de categóricas

# Nulos por columna
nulos = df.isnull().sum()
nulos_pct = (nulos / len(df)) * 100
print(pd.DataFrame({'Nulos': nulos, '%': nulos_pct})[nulos > 0])

# Duplicados
print(f'Duplicados: {df.duplicated().sum()}')
df = df.drop_duplicates().reset_index(drop=True)

# Distribuciones numéricas
df.hist(figsize=(15,10), bins=20)
plt.tight_layout()
plt.show()

# Correlaciones
sns.heatmap(df.select_dtypes(include=np.number).corr(),
            annot=True, fmt='.2f', cmap='coolwarm')
```

4.2 Tipos de datos faltantes (Missing Data)

Entender el mecanismo de generación de los valores faltantes es crucial para elegir la estrategia de imputación correcta:

Tipo	Descripción	Ejemplo	Estrategia recomendada
------	-------------	---------	------------------------

MCAR (Missing Completely At Random)	La ausencia no depende de ninguna variable. Patrón completamente aleatorio.	Fallo técnico del sensor, error de entrada	Eliminación de filas o imputación simple (media/mediana/moda)
MAR (Missing At Random)	La ausencia depende de otras variables observadas, no del valor faltante en sí.	Hombres responden menos la variable 'peso' en una encuesta	Imputación múltiple (MICE), KNN Imputer, regresión
MNAR (Missing Not At Random)	La ausencia depende del propio valor faltante (el más problemático).	Personas con ingresos muy altos no reportan su salario	Análisis específico del negocio, flag de missingness como variable

4.3 Estrategias de imputación

```
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# 1. Imputación estadística (MCAR):
df['edad'].fillna(df['edad'].median(), inplace=True) # mediana: robusta a outliers
df['altura'].fillna(df['altura'].mean(), inplace=True) # media: si dist. es simétrica
df['plan'].fillna(df['plan'].mode()[0], inplace=True) # moda: variables categóricas

# 2. KNN Imputer (MAR - usa relaciones entre variables):
knn_imp = KNNImputer(n_neighbors=5)
df_imp = pd.DataFrame(knn_imp.fit_transform(df_num), columns=df_num.columns)

# 3. MICE - Múltiple Imputation by Chained Equations (MAR complejo):
mice = IterativeImputer(max_iter=10, random_state=42)
df_mice = pd.DataFrame(mice.fit_transform(df_num), columns=df_num.columns)

# REGLA: fit() SOLO sobre train set. transform() sobre train Y test.
# mice.fit(X_train)
# X_train_imp = mice.transform(X_train)
# X_test_imp = mice.transform(X_test) # usa estadísticos de train
```

4.4 Detección y tratamiento de outliers

Los outliers son valores que se desvían significativamente del patrón general. Pueden ser errores de medición o datos legítimos extremos. La decisión de tratar un outlier debe basarse siempre en el contexto del negocio.

```
import numpy as np
from scipy import stats

# — Método 1: Z-Score (asume distribución normal) —
z_scores = np.abs(stats.zscore(df['edad']))
outliers_z = df[z_scores > 3] # > 3 std desviaciones
print(f'Outliers Z-score: {len(outliers_z)}')
```

```
# — Método 2: IQR (no paramétrico, más robusto) —
Q1 = df['peso'].quantile(0.25)
Q3 = df['peso'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Visualizar:
plt.boxplot(df['peso'])
outliers_iqr = df[(df['peso'] < lower) | (df['peso'] > upper)]
print(f'Outliers IQR: {len(outliers_iqr)}')

# — Método 3: Isolation Forest (multivariado) —
from sklearn.ensemble import IsolationForest
if_model = IsolationForest(contamination=0.05, random_state=42)
df['outlier_if'] = if_model.fit_predict(df[num_cols])
# -1 = outlier, 1 = normal

# — Método 4: LOF - Local Outlier Factor —
from sklearn.neighbors import LocalOutlierFactor
lof = LocalOutlierFactor(n_neighbors=20)
df['outlier_lof'] = lof.fit_predict(df[num_cols])

# — Tratamientos posibles —
# a) Eliminación: df = df[z_scores <= 3]
# b) Winsorizing (capping al percentil 99):
from scipy.stats import mstats
df['peso_wins'] = mstats.winsorize(df['peso'], limits=[0.01, 0.01])
# c) Imputar con mediana (tratar como missing)
df.loc[z_scores > 3, 'edad'] = df['edad'].median()
```

4.5 Codificación de variables categóricas

Técnica	Cuándo usarla	Código	Ventajas / Desventajas
One-Hot Encoding	Categórica nominal sin orden, pocas categorías (< 15)	pd.get_dummies(df['plan']) OneHotEncoder()	Sin orden artificial. Crea columnas dispersas (alta cardinalidad = muchas columnas)
Label Encoding	Categórica ordinal (con orden real)	LabelEncoder().fit_transform(df['nivel'])	Compacto. Introduce orden artificial no deseado en variables nominales
Target Encoding	Alta cardinalidad (ciudades, códigos postales)	df.groupby('ciudad')['target'].mean()	Captura información del target. Riesgo de data leakage → usar solo en train
Ordinal Encoding	Categórica con orden explícito	OrdinalEncoder(categories=[['bajo', 'medio', 'alto']])	Preserva el orden. Solo válido si el orden es real
Frequency Encoding	Alta cardinalidad, frecuencia es informativa	df['ciudad'].map(df['ciudad'].value_counts())	Simple y efectivo. No introduce muchas columnas

4.6 Escalado y normalización de variables numéricas

Los algoritmos basados en distancias (KNN, SVM, K-Means) y los modelos de redes neuronales son muy sensibles a la escala de las variables. Es imprescindible escalar antes de entrenar estos modelos. Los árboles de decisión y derivados (Random Forest, XGBoost) son invariantes al escalado.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

# — MinMaxScaler: normalización al rango [0, 1] —
# Fórmula:  $(x - \min) / (\max - \min)$ 
# Cuándo: cuando se conoce el rango máximo y mínimo, redes neuronales, imágenes
# Debilidad: muy sensible a outliers
scaler_mm = MinMaxScaler()
X_norm = scaler_mm.fit_transform(X_train) # fit + transform sobre train

# — StandardScaler: estandarización (media=0, std=1) —
# Fórmula:  $(x - \mu) / \sigma$ 
# Cuándo: SVM, PCA, regresión, la mayoría de casos generales
# Debilidad: sensible a outliers
scaler_std = StandardScaler()
X_std = scaler_std.fit_transform(X_train)

# — RobustScaler: basado en cuartiles —
# Fórmula:  $(x - Q2) / IQR$ 
# Cuándo: cuando hay outliers que no se pueden eliminar
# Fortaleza: robusto a outliers extremos
scaler_rob = RobustScaler()
X_rob = scaler_rob.fit_transform(X_train)

# ⚠ REGLA CRÍTICA ANTI-LEAKAGE:
# scaler.fit() SOLO sobre X_train
# scaler.transform() sobre X_train Y X_test
# NUNCA: scaler.fit_transform(X) # sobre todo el dataset = DATA LEAKAGE
```

Scaler	Fórmula	Sensible a outliers	Mejor para
MinMaxScaler	$(x - \min) / (\max - \min)$	Sí, muy sensible	Redes neuronales, imágenes, datos acotados
StandardScaler	$(x - \mu) / \sigma$	Moderadamente	SVM, PCA, regresión logística, uso general
RobustScaler	$(x - Q2) / IQR$	No	Datos con outliers que no se deben eliminar

4.7 Data Leakage: el error más peligroso en ML

El data leakage ocurre cuando información del conjunto de test se filtra al proceso de entrenamiento, produciendo modelos con métricas artificialmente infladas que fallan en producción. Es el error más común y devastador en proyectos de ML.

Tipo de leakage	Descripción	Ejemplo
-----------------	-------------	---------

Temporal leakage	Usar información futura para predecir el pasado	Incluir el precio de mañana para predecir si el cliente comprará hoy
Preprocessing leakage	Aplicar fit() sobre todo el dataset antes de dividir	scaler.fit(X_total) antes de train/test split
Target leakage	Variable predictora que solo se conoce después del evento objetivo	Incluir 'días_de_tratamiento' para predecir si el paciente fue diagnosticado
Group leakage	Misma entidad en train y test (usuarios, pacientes)	El mismo usuario aparece en train y en test → el modelo memoriza al usuario

```
# — Patrón correcto: Pipeline de sklearn —
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# El Pipeline garantiza que fit() solo ocurre sobre train
numerical_features = ['edad', 'peso', 'altura']
categorical_features = ['genero', 'plan', 'ciudad']

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
])

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
])

# fit() solo sobre train → CERO leakage
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test) # preprocesamiento automático sobre test
```

4.8 Caso práctico: Pipeline GymPro (gimnasio_gympro.csv)

El dataset de GymPro contiene 308 registros de socios de un gimnasio con 15 columnas: ID_Socio, Nombre, Genero, Edad, Altura_cm, Peso_kg, Ciudad, Plan, Cuota_mensual_EUR, Antigüedad_meses, Dias_por_semana, Minutos_por_sesion, Calorias_quemadas, Satisfaccion, Fecha_inscripcion. Presentaba los siguientes problemas de calidad:

- 7 filas duplicadas exactas.
- Valores nulos en 7 columnas (Edad, Altura_cm, Peso_kg, Calorias_quemadas, Satisfaccion, Plan, Ciudad).
- Categorías inconsistentes: 'masculino', 'M', 'm', 'femenino', 'F', 'f' para la variable Genero.
- Outliers extremos: Edad=220 años, Altura=250 cm (errores de entrada).

```

def preprocesar_dataset_gimnasio(ruta_csv: str) -> pd.DataFrame:
    """Pipeline completo de preprocesamiento para GymPro."""
    # 1. Carga
    df = pd.read_csv(ruta_csv)
    print(f'Shape inicial: {df.shape}') # (308, 15)

    # 2. Eliminar duplicados
    df = df.drop_duplicates().reset_index(drop=True) # → 301 filas

    # 3. Limpiar categóricas inconsistentes
    df['Genero'] = (df['Genero'].str.strip().str.lower()
                    .map({'m':'M', 'f':'F', 'masculino':'M', 'femenino':'F'}))
    df['Plan'] = df['Plan'].str.strip().str.capitalize()
    df['Ciudad'] = df['Ciudad'].str.strip().str.capitalize()

    # 4. Imputar nulos
    df['Edad'] = df['Edad'].fillna(df['Edad'].median())
    df['Altura_cm'] = df['Altura_cm'].fillna(df['Altura_cm'].mean())
    df['Peso_kg'] = df['Peso_kg'].fillna(df['Peso_kg'].median())
    df['Satisfaccion'] = df['Satisfaccion'].fillna(df['Satisfaccion'].mode()[0])

    # 5. Tratar outliers con IQR (capping)
    for col in ['Edad', 'Altura_cm', 'Peso_kg', 'Calorias_quemadas']:
        Q1, Q3 = df[col].quantile([0.25, 0.75])
        IQR = Q3 - Q1
        lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
        df[col] = df[col].clip(lower, upper) # capping en lugar de eliminación

    # 6. Estandarizar (añadir columnas _z)
    from sklearn.preprocessing import StandardScaler
    num_cols = ['Edad', 'Altura_cm', 'Peso_kg', 'Antiguedad_meses',
                'Dias_por_semana', 'Minutos_por_sesion', 'Calorias_quemadas']
    scaler = StandardScaler()
    z_values = scaler.fit_transform(df[num_cols])
    for i, col in enumerate(num_cols):
        df[f'{col}_z'] = z_values[:, i]

    print(f'Shape final: {df.shape}') # (301, 22)
    return df

df_clean = preprocesar_dataset_gimnasio('gimnasio_gympro.csv')

```

5. MODELADO Y EVALUACIÓN DE MODELOS

Una vez los datos están preprocesados, la fase de modelado consiste en seleccionar el algoritmo adecuado al problema, entrenarlo sobre los datos de entrenamiento, evaluar su rendimiento y optimizar sus hiperparámetros. La evaluación honesta de un modelo requiere métricas adecuadas al tipo de problema y una división correcta de los datos.

5.1 El mapa de algoritmos de scikit-learn

La selección del algoritmo debe guiarse por el tipo de problema, el tamaño del dataset y las restricciones de interpretabilidad y velocidad:

Problema	Dataset pequeño (<1K)	Dataset mediano (1K-100K)	Dataset grande (>100K)
Regresión	Ridge/Lasso Regression, SVR	Random Forest, Gradient Boosting (XGBoost)	SGD Regressor, Neural Networks
Clasificación binaria	Logistic Regression, SVM	Random Forest, XGBoost, LightGBM	Neural Networks, LightGBM
Clasificación multiclase	Naive Bayes, SVM	Random Forest, XGBoost	Neural Networks (softmax)
Clustering	K-Means (k pequeño), DBSCAN	K-Means, MiniBatch K-Means	MiniBatch K-Means, BIRCH

5.2 El ritual fit/predict de scikit-learn

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# División estratificada (mantiene proporciones de clases)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Entrenamiento
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predicción
y_pred = model.predict(X_test) # clases predichas
y_proba = model.predict_proba(X_test) # probabilidades (para AUC-ROC)

# Importancia de características
importances = pd.Series(model.feature_importances_, index=X.columns)
importances.sort_values(ascending=False).plot(kind='bar')
```

5.3 Métricas de evaluación para Regresión

Métrica	Fórmula	Interpretación	Cuándo usarla
MAE (Mean Absolute Error)	$\text{mean}(y - \hat{y})$	Error promedio en las mismas unidades del target. Robusto a outliers.	Cuando todos los errores son igual de importantes
MSE (Mean Squared Error)	$\text{mean}((y - \hat{y})^2)$	Penaliza errores grandes	Cuando los errores grandes son

		desproporcionadamente.	especialmente costosos
RMSE (Root MSE)	$\sqrt{\text{MSE}}$	Mismas unidades que el target. Más interpretable que MSE.	Uso general; permite comparar con la desviación estándar del target
R ² (R-cuadrado)	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Proporción de varianza explicada. 1 = perfecto, 0 = basal, <0 = peor que la media.	Para comunicar el rendimiento del modelo de forma intuitiva

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f'MAE: {mae:.2f}')
print(f'RMSE: {rmse:.2f}')
print(f'R2: {r2:.4f}')

```

5.4 Métricas de evaluación para Clasificación

La matriz de confusión es la base de todas las métricas de clasificación. Para un problema binario (clase positiva vs. negativa):

Métrica	Fórmula	Interpretación práctica
Accuracy	$(\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$	Proporción de predicciones correctas. Engañosa con clases desbalanceadas.
Precision	$\text{TP} / (\text{TP} + \text{FP})$	De todos los que predijo como positivos, ¿cuántos lo eran realmente? Importante cuando FP es costoso (spam).
Recall (Sensibilidad)	$\text{TP} / (\text{TP} + \text{FN})$	De todos los positivos reales, ¿cuántos detectó? Importante cuando FN es costoso (detección de cáncer).
F1-Score	$2 \cdot (\text{P} \cdot \text{R}) / (\text{P} + \text{R})$	Media armónica de Precision y Recall. Equilibra ambas métricas. Para datasets desbalanceados.
AUC-ROC	Área bajo curva ROC	Probabilidad de que el modelo clasifique un positivo por encima de un negativo. 0.5 = aleatorio, 1.0 = perfecto.
AUC-PR	Área bajo curva P-R	Mejor que AUC-ROC para clases muy desbalanceadas.

```

from sklearn.metrics import (
    classification_report, confusion_matrix,

```

```

    roc_auc_score, f1_score, precision_score, recall_score
)

# Reporte completo
print(classification_report(y_test, y_pred))

# Matriz de confusión
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred Neg', 'Pred Pos'],
            yticklabels=['Real Neg', 'Real Pos'])

# AUC-ROC (requiere probabilidades)
auc = roc_auc_score(y_test, y_proba[:, 1])
print(f'AUC-ROC: {auc:.4f}')

```

5.5 Overfitting, Underfitting y Bias-Variance Tradeoff

El error de un modelo se descompone en tres componentes: Bias (sesgo) + Variance (varianza) + Ruido irreducible. La gestión de este equilibrio es fundamental en el diseño de modelos de ML.

Problema	Síntoma	Causa	Solución
Underfitting (alto bias)	Error alto tanto en train como en test. El modelo es demasiado simple.	Modelo con poca capacidad, pocas iteraciones, features insuficientes.	Modelo más complejo, más features, menos regularización, más épocas.
Overfitting (alta varianza)	Error muy bajo en train, alto en test. El modelo memoriza los datos.	Modelo demasiado complejo, pocos datos de entrenamiento.	Más datos, regularización (L1/L2), Dropout, Early Stopping, validación cruzada.
Buen ajuste	Error bajo y similar en train y test.	Balance adecuado entre complejidad y datos.	Mantener. Monitorizar en producción.

5.6 Validación cruzada (Cross-Validation)

La validación cruzada es la técnica estándar para obtener una estimación robusta del rendimiento del modelo sin desperdiciar datos. K-Fold divide el dataset en k partes iguales, entrena k veces (cada vez usando k-1 partes para entrenar y 1 para validar) y promedia los resultados.

```

from sklearn.model_selection import (
    cross_val_score, StratifiedKFold, TimeSeriesSplit
)

# — K-Fold estándar —
scores = cross_val_score(model, X, y, cv=5, scoring='f1_weighted')
print(f'F1 CV: {scores.mean():.3f} ± {scores.std():.3f}')

# — Stratified K-Fold (preserva proporciones de clases) —
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):

```

```

X_tr, X_val = X.iloc[train_idx], X.iloc[val_idx]
y_tr, y_val = y.iloc[train_idx], y.iloc[val_idx]
# entrenar y evaluar...

# — Time Series Split (para datos temporales) —
# Respetar el orden temporal. Nunca usar K-Fold aleatorio con series temporales.
tscv = TimeSeriesSplit(n_splits=5)
for train_idx, test_idx in tscv.split(X):
    # train siempre ANTES que test en el tiempo
    pass

```

5.7 Optimización de hiperparámetros

Los hiperparámetros son los parámetros del modelo que se fijan antes del entrenamiento (no se aprenden de los datos). Su optimización es crucial para maximizar el rendimiento del modelo.

```

from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from scipy.stats import randint, uniform

# — Grid Search (búsqueda exhaustiva) —
# Evalúa TODAS las combinaciones del grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 20, None],
    'min_samples_split': [2, 5, 10],
    'max_features': ['sqrt', 'log2']
}
# 3 × 4 × 3 × 2 = 72 combinaciones × 5 folds = 360 entrenamientos
grid_cv = GridSearchCV(RandomForestClassifier(random_state=42),
                        param_grid, cv=5, scoring='f1_weighted',
                        n_jobs=-1, verbose=1)
grid_cv.fit(X_train, y_train)
print(f'Mejores parámetros: {grid_cv.best_params_}')
print(f'Mejor F1 CV: {grid_cv.best_score_:.4f}')

# — Random Search (muestreo aleatorio - más eficiente) —
param_dist = {
    'n_estimators': randint(50, 500),
    'max_depth': randint(3, 30),
    'min_samples_split': randint(2, 20),
    'max_features': uniform(0.1, 0.9)
}
random_cv = RandomizedSearchCV(RandomForestClassifier(random_state=42),
                                param_dist, n_iter=100, cv=5,
                                scoring='f1_weighted', random_state=42)
random_cv.fit(X_train, y_train)

# Mejor estimador listo para producción:
best_model = random_cv.best_estimator_

```

5.8 Regularización para evitar overfitting

Técnica	Dónde se aplica	Efecto	Hiperparámetro
L1 (Lasso)	Regresión lineal/logística	Pone algunos coeficientes a	α (alpha)

		exactamente 0 → selección automática de features.	
L2 (Ridge)	Regresión lineal/logística	Reduce los coeficientes hacia 0 pero no exactamente a 0. Estable con multicolinealidad.	α (alpha)
ElasticNet	Regresión	Combinación de L1 y L2. Balanceable.	α , l1_ratio
Dropout	Redes neuronales	Desactiva aleatoriamente un porcentaje de neuronas durante el entrenamiento. Previene co-adaptación.	rate (típico: 0.2-0.5)
Early Stopping	Redes neuronales, Boosting	Detiene el entrenamiento cuando la métrica de validación deja de mejorar.	patience
Batch Normalization	Redes neuronales profundas	Normaliza las activaciones de cada capa. Estabiliza el entrenamiento, permite learning rates más altos.	momentum, epsilon

6. MLOPS: IMPLEMENTACIÓN, DESPLIEGUE Y MONITORIZACIÓN

MLOps (Machine Learning Operations) es la disciplina que combina principios de DevOps con las particularidades del ciclo de vida del Machine Learning. Su objetivo es llevar modelos a producción de forma fiable, reproducible y escalable, y mantenerlos funcionando correctamente a lo largo del tiempo.

6.1 El ciclo de vida del modelo ML

A diferencia del software tradicional, los modelos de ML tienen características únicas que complican su operación en producción: dependen de los datos (que cambian con el tiempo), son probabilísticos, y su rendimiento puede degradarse silenciosamente sin errores de código.

Fase	Actividades	Herramientas
Desarrollo (DEV)	Exploración, experimentación, prototipado de modelos. Acceso a datos sensibles limitado.	Jupyter, MLflow, DVC, Git
Validación (TEST/STAGING)	Evaluación rigurosa, tests de rendimiento, integración con sistemas downstream. Datos anonimizados.	pytest, Great Expectations, MLflow
Producción (PROD)	Servir predicciones a usuarios reales, monitorización continua, reentrenamiento.	Docker, Kubernetes, Prometheus, Grafana

6.2 Niveles de madurez MLOps

Nivel	Descripción	Características	Cuándo adoptarlo
Nivel 0 (Manual)	Proceso completamente manual. El científico de datos hace todo a mano.	Código en notebooks, despliegue manual, sin CI/CD, sin monitorización.	Proyectos piloto, proof of concept
Nivel 1 (Automatización de entrenamiento)	Pipeline de ML automatizado. El entrenamiento se puede disparar automáticamente.	Pipelines de datos/entrenamiento automatizados, seguimiento de experimentos (MLflow), versionado de modelos.	Producción inicial, cuando hay 1-3 modelos activos
Nivel 2 (CI/CD completo)	CI/CD/CT completo. Los cambios en el código o en los datos disparan automáticamente el reentrenamiento y redespliegue.	CI/CD/CT pipelines, A/B testing automatizado, monitorización avanzada, feature store.	Organizaciones con muchos modelos en producción, alta cadencia de cambio

6.3 CI/CD/CT en MLOps

Los tres pilares de la automatización en MLOps:

- **CI – Integración Continua:** Cada commit activa tests automáticos: tests unitarios del código de preprocesamiento, tests de calidad de datos (Great Expectations), tests de rendimiento del modelo con datos de referencia.
- **CD – Despliegue Continuo:** El modelo validado se empaqueta en un contenedor Docker y se despliega automáticamente en el entorno de staging/producción sin intervención manual.
- **CT – Entrenamiento Continuo:** Exclusivo de MLOps. Cuando el rendimiento del modelo cae por debajo de un umbral o llega suficiente dato nuevo, se dispara automáticamente un nuevo ciclo de entrenamiento.

6.4 Contenerización: Docker y Kubernetes

```
# — Dockerfile para servir un modelo de ML —
FROM python:3.11-slim

WORKDIR /app

# Copiar y instalar dependencias
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar código del modelo y artefactos
COPY model.pkl .
COPY app.py .

# Variables de entorno
ENV MODEL_VERSION=1.2.3
ENV LOG_LEVEL=INFO

# Puerto de la API
EXPOSE 8080

CMD ["python", "app.py"]

# — API REST con FastAPI —
from fastapi import FastAPI
import joblib
import pandas as pd

app = FastAPI()
model = joblib.load('model.pkl')

@app.post('/predict')
def predict(data: dict):
    df = pd.DataFrame([data])
    prediction = model.predict(df)[0]
    proba = model.predict_proba(df)[0].max()
    return {'prediction': int(prediction), 'confidence': float(proba)}
```

Kubernetes orquesta múltiples contenedores Docker en producción, proporcionando: auto-scaling (más réplicas cuando aumenta la carga), auto-healing (reinicia contenedores caídos), rolling updates (actualiza el modelo sin downtime) y load balancing.

6.5 Plataformas Cloud de MLOps

Plataforma	Proveedor	Fortalezas	Uso típico
AWS SageMaker	Amazon Web Services	Ecosistema completo: labeling, entrenamiento distribuido, despliegue, monitorización. Integración nativa con S3, Lambda.	Empresas ya en AWS, alto volumen de datos
Azure Machine Learning	Microsoft Azure	Integración con herramientas Microsoft (Power BI, DevOps). MLOps maduros. Designer visual para no-coders.	Empresas con ecosistema Microsoft/Azure
GCP Vertex AI	Google Cloud Platform	AutoML potente, BigQuery ML integrado, TPUs para entrenamiento de DL. Pipelines Kubeflow nativos.	Uso intensivo de BigQuery, modelos de DL complejos
MLflow (open source)	Independiente	Tracking de experimentos, registro de modelos, despliegue. Self-hosted o en cualquier cloud.	Independencia del proveedor, proyectos open source

6.6 Monitorización de modelos en producción

Un modelo que funciona perfectamente el día del despliegue puede degradarse progresivamente sin que haya ningún error de software. La monitorización detecta estos problemas antes de que impacten en el negocio.

Tipo de degradación	Descripción	Ejemplo	Detección
Data Drift (deriva de datos)	La distribución de las variables de entrada cambia respecto al training set.	Una tienda online empieza a recibir clientes de un nuevo segmento demográfico no visto en entrenamiento.	Test estadístico (KS-test, Chi-cuadrado) sobre las distribuciones de features. PSI (Population Stability Index).
Concept Drift (deriva de concepto)	La relación entre variables de entrada y el target cambia con el tiempo.	Un modelo de predicción de fraude entrenado antes de una nueva técnica de fraude desconocida.	Monitorización de métricas de rendimiento (MAE, F1) sobre nuevos datos etiquetados.
Label Drift (deriva de etiquetas)	La distribución del target cambia (sin cambio en las features).	La proporción de clientes que hacen churn aumenta estacionalmente.	Análisis de la distribución de predicciones y etiquetas reales en ventanas temporales.

6.7 Estrategias de reentrenamiento

- **Reentrenamiento por calendario (fixed schedule):** El modelo se reentrena cada semana/mes/trimestre independientemente del rendimiento. Simple de implementar. Puede reentrenar innecesariamente o tarde.
- **Reentrenamiento por umbral de rendimiento (trigger dinámico):** El reentrenamiento se dispara cuando el rendimiento cae por debajo de un umbral definido. Requiere etiquetas en tiempo real o evaluación online. Más eficiente.
- **Híbrido:** Combinación: evaluación continua + ventana de tiempo mínima entre reentrenamientos para evitar oscilaciones.

6.8 Experimento de referencia: Netflix

Netflix es el caso de referencia en MLOps a escala. Su sistema de recomendación es un modelo de ML que personaliza los thumbnails (imágenes de portada) de cada contenido según el usuario. El mismo contenido muestra imágenes diferentes a usuarios distintos basándose en su historial de visualización. El sistema realiza A/B testing continuo de miles de variantes de modelos simultáneamente, con rollback automático si una variante nueva no mejora las métricas clave (tasa de clic, tiempo de reproducción). Este enfoque muestra cómo el MLOps de Nivel 2 permite innovación continua sin riesgo de degradar la experiencia de usuario.

7. APLICACIONES EN CIENCIA DEL COMPORTAMIENTO Y CASOS DE USO

La Ciencia del Comportamiento combina psicología, economía y ciencia de datos para entender y predecir el comportamiento humano. El ML permite escalar estos análisis a millones de observaciones, identificando patrones que serían imposibles de detectar manualmente.

7.1 Segmentación de clientes con RFM + K-Means

El análisis RFM (Recency, Frequency, Monetary Value) es la técnica de segmentación de clientes más utilizada en marketing y retail. Combina tres dimensiones del comportamiento de compra para clasificar clientes según su valor y lealtad.

Dimensión	Definición	Cálculo	Significado de valor alto
Recency (R)	Días desde la última compra	fecha_hoy - fecha_última_compra	Cliente activo, compró recientemente
Frequency (F)	Número de transacciones en el periodo	count(order_id) por cliente	Cliente fiel, compra frecuentemente
Monetary Value (M)	Gasto total en el periodo	sum(valor_compra) por cliente	Cliente de alto valor económico

Dataset de referencia: Online Retail UCI (Kaggle). 541.909 transacciones del período 2010-2011, 4.373 clientes únicos del Reino Unido.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# — Carga y limpieza —
df = pd.read_csv('online_retail.csv', encoding='latin-1')
df = df.dropna(subset=['CustomerID'])
df = df[df['Quantity'] > 0]
df = df[df['UnitPrice'] > 0]
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
df['TotalPrice'] = df['Quantity'] * df['UnitPrice']

# — Cálculo RFM —
reference_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

rfm = df.groupby('CustomerID').agg(
    Recency = ('InvoiceDate', lambda x: (reference_date - x.max()).days),
    Frequency = ('InvoiceNo', 'nunique'), # n° de facturas únicas
    Monetary = ('TotalPrice', 'sum')
).reset_index()

print(rfm.describe())

# — Normalización (K-Means es sensible a escala) —
```

```

scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])

# — Elbow Method para seleccionar k —
inertias = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(rfm_scaled)
    inertias.append(km.inertia_)

plt.plot(range(2, 11), inertias, 'bo-')
plt.xlabel('Número de clusters k')
plt.ylabel('Inercia')
plt.title('Elbow Method - Selección de k')
plt.axvline(x=4, color='r', linestyle='--', label='k=4 óptimo')

# — Segmentación final con k=4 —
km = KMeans(n_clusters=4, random_state=42, n_init=10)
rfm['Cluster'] = km.fit_predict(rfm_scaled)

# — Interpretación de segmentos —
resumen = rfm.groupby('Cluster').agg(
    n_clientes = ('CustomerID', 'count'),
    R_media    = ('Recency', 'mean'),
    F_media    = ('Frequency', 'mean'),
    M_media    = ('Monetary', 'mean')
).round(1)
print(resumen)

# Ejemplo de interpretación de clusters:
# Cluster 0: R bajo, F alto, M alto → Champions (mejores clientes)
# Cluster 1: R alto, F bajo, M bajo → Lost customers (inactivos)
# Cluster 2: R bajo, F medio, M medio → Loyal customers
# Cluster 3: R medio, F bajo, M medio → At risk (riesgo de pérdida)

```

7.2 Predicción de series temporales

Las series temporales son secuencias de datos indexadas en el tiempo. Su análisis y predicción requiere técnicas especiales que respeten el orden temporal de las observaciones.

Técnica	Tipo de modelo	Cuándo usarla
ARIMA/SARIMA	Estadístico	Series univariadas con tendencia y estacionalidad. Alta interpretabilidad.
Prophet (Meta)	Estadístico avanzado	Series con múltiples estacionalidades, festivos. Fácil de configurar.
LSTM/GRU	Deep Learning	Dependencias temporales complejas, múltiples variables. Requiere más datos.
Transformer (Temporal Fusion Transformer)	Deep Learning	Series largas con múltiples covariables y diferentes frecuencias.

LightGBM/XGBoost + feature engineering temporal	ML clásico	Enfoque práctico con lag features, rolling stats. Muy efectivo en competiciones.
---	------------	--

```
# — Feature engineering temporal para ML —
import pandas as pd

df['fecha'] = pd.to_datetime(df['fecha'])
df = df.sort_values('fecha')

# Features temporales básicas
df['año'] = df['fecha'].dt.year
df['mes'] = df['fecha'].dt.month
df['dia_sem'] = df['fecha'].dt.dayofweek # 0=lunes
df['trimestre'] = df['fecha'].dt.quarter
df['es_fin_semana'] = (df['dia_sem'] >= 5).astype(int)

# Lag features (valores pasados)
for lag in [1, 7, 14, 30]:
    df[f'lag_{lag}'] = df['target'].shift(lag)

# Rolling statistics (estadísticas en ventana temporal)
for window in [7, 30]:
    df[f'roll_mean_{window}'] = df['target'].rolling(window).mean()
    df[f'roll_std_{window}'] = df['target'].rolling(window).std()
    df[f'roll_max_{window}'] = df['target'].rolling(window).max()

# Eliminar filas con NaN introducidos por lags
df = df.dropna()

# ⚠ TimeSeriesSplit: NUNCA K-Fold aleatorio con series temporales
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5, gap=7) # gap=7 días entre train y val
```

7.3 ML en Psicología y Salud Mental

La IA está transformando la Ciencia del Comportamiento y la Psicología clínica con aplicaciones que complementan (no sustituyen) al profesional humano:

- **Detección temprana de salud mental:** Modelos de clasificación sobre texto (NLP) para detectar indicadores lingüísticos asociados a depresión, ansiedad y riesgo de suicidio en redes sociales o apps de salud mental.
- **Personalización de intervenciones terapéuticas:** Sistemas de recomendación que adaptan el contenido terapéutico según el progreso y preferencias del paciente, aumentando la adherencia al tratamiento.
- **Predicción de recaídas:** Modelos predictivos sobre datos de comportamiento (apps, wearables) para anticipar recaídas en trastornos como el TOC, el TLP o las adicciones.
- **Psicoterapia asistida por IA:** ChatBots terapéuticos con NLP (Woebot, Wysa) como primera línea de atención y escalado al profesional cuando es necesario.

Consideraciones éticas críticas en este dominio: el consentimiento informado debe especificar el uso de IA, los modelos deben ser auditados para sesgos (grupos étnicos,

géneros), los datos clínicos requieren el máximo nivel de protección (GDPR Art.9), y la IA debe actuar siempre como asistente del profesional, nunca como sustituto.

8. STACK TECNOLÓGICO PROFESIONAL DE ML

El ecosistema de herramientas para ML es extenso. Este stack de referencia cubre las necesidades de la mayoría de proyectos profesionales, desde la exploración de datos hasta el despliegue y monitorización en producción.

Capa	Herramienta	Propósito	Cuándo usar
Datos	pandas	Manipulación y análisis de datos tabulares en memoria	Siempre. Dataset < RAM disponible.
Datos	numpy	Computación numérica eficiente. Base de scikit-learn y TensorFlow.	Siempre (dependencia de casi todo).
Datos	Apache Spark / PySpark	Procesamiento distribuido de Big Data. Escala a petabytes.	Dataset > RAM del nodo único.
Visualización	matplotlib	Visualizaciones estáticas de bajo nivel. Control total.	Gráficos publicación científica.
Visualización	seaborn	Gráficos estadísticos sobre matplotlib. API más simple.	EDA, análisis estadístico.
Visualización	plotly	Gráficos interactivos para dashboards y Jupyter.	Exploración interactiva, presentaciones.
ML clásico	scikit-learn	Algoritmos de ML, preprocesamiento, evaluación, pipelines.	ML clásico. Estándar de la industria.
ML clásico	XGBoost / LightGBM	Gradient Boosting de alto rendimiento. Ganador de competiciones.	Datos tabulares estructurados.
Deep Learning	TensorFlow + Keras	Redes neuronales. API Keras de alto nivel. Producción robusta.	Producción en empresas grandes.
Deep Learning	PyTorch	Redes neuronales. Más flexible que TF. Estándar en investigación.	Investigación, NLP avanzado (HuggingFace).
MLOps	MLflow	Tracking de experimentos, registro y versionado de modelos.	Siempre que haya > 1 experimento.
MLOps	DVC	Versionado de datos y pipelines (Git para datos).	Cuando los datos cambian frecuentemente.
MLOps	Kubeflow	Pipelines de ML sobre Kubernetes.	MLOps Nivel 2 en Kubernetes.
Calidad de datos	Great Expectations	Tests automatizados de calidad y contratos de datos.	Pipelines de datos en producción.
Despliegue	FastAPI	API REST para servir predicciones del modelo.	Servicio de inferencia en producción.

Despliegue	Docker	Contenerización del modelo y sus dependencias.	Siempre en producción.
Despliegue	Kubernetes	Orquestación, escalado y gestión de contenedores.	Producción a escala.
Monitorización	Prometheus + Grafana	Métricas de rendimiento del sistema y del modelo.	Monitorización de producción.
Monitorización	Evidently AI	Drift detection y reportes de calidad del modelo.	Monitorización específica de ML.

9. CHECKLIST PROFESIONAL: PROYECTO DE ML DE EXTREMO A EXTREMO

Esta checklist recoge las prácticas profesionales imprescindibles en cada fase de un proyecto de Machine Learning. Úsala como guía de calidad antes de presentar resultados o desplegar un modelo.

FASE 1: Definición del problema

- ¿Es realmente un problema de ML o puede resolverse con reglas simples?
- ¿Está la variable objetivo (target) claramente definida y disponible?
- ¿Cuál es la métrica de negocio que se quiere optimizar?
- ¿Se dispone de suficientes datos históricos (mínimo ~1.000-10.000 ejemplos para ML clásico)?
- ¿Se ha definido el baseline (modelo de referencia simple)?

FASE 2: Datos

- EDA completo: distribuciones, nulos, duplicados, outliers, correlaciones.
- El tipo de datos faltantes está identificado (MCAR/MAR/MNAR).
- La estrategia de imputación es apropiada al mecanismo de missingness.
- Los outliers han sido investigados (error vs. dato legítimo extremo).
- Las variables categóricas están codificadas apropiadamente.
- Todas las transformaciones se fitean solo sobre el conjunto de entrenamiento.

FASE 3: Modelado

- División train/val/test correcta y estratificada.
- Se ha establecido y medido un modelo baseline.
- Las métricas son apropiadas al problema (no usar accuracy con clases desbalanceadas).
- Se usa validación cruzada para estimaciones robustas.
- La búsqueda de hiperparámetros usa cross-validation (no el test set).
- El test set se usa UNA SOLA VEZ para la evaluación final.

FASE 4: Despliegue y MLOps

- El modelo está serializado junto con su pipeline de preprocesamiento (joblib/pickle).
- La versión del modelo y sus métricas están registradas en MLflow.
- El servicio de inferencia tiene tests de integración automatizados.
- Hay monitorización de data drift y concept drift activa en producción.
- Existe una estrategia de reentrenamiento documentada.
- Hay un plan de rollback ante degradación del modelo.
- Los datos usados en producción cumplen GDPR (si aplica).